Semana 2 – GitHub, CTEs, Window Functions y EDA

> Control de versiones con GitHub, CTEs encadenados, Window Functions (ROW_NUMBER, RANK, LAG, LEAD), Unity Catalog y EDA para Data Engineering.

Tabla de contenidos

1. GitHub y Control de Versiones integrado con Databricks
2. CTEs: WITH para código limpio y reutilizable
3. Window Functions: análisis sin perder granularidad
4. PARTITION BY vs GROUP BY
5. Unity Catalog y Tablas Managed vs External
6. EDA para Data Engineering: los 6 pasos

---

1. GitHub y Control de Versiones integrado con Databricks

Git es el sistema de control de versiones local. **GitHub** es la plataforma en la nube para hospedar repositorios.

Conceptos clave: - **Repositorio:** carpeta del proyecto con todo el historial de cambios - **Commit:** guardado de cambios con un mensaje descriptivo. Separar en commits pequeños con mensajes claros - **Branch (rama):** línea de desarrollo paralela. Se crean **features** para trabajar sin afectar el código principal - **Pull Request (PR):** solicitud para meter cambios en el repo. Alguien debe **aprobar** el PR antes de hacer merge - **Merge conflicts:** ocurren cuando dos personas modifican el mismo archivo

Flujo master/developer: las branches se mergean primero a `developer` para probar, y solo después a `master` (producción).

Integración con Databricks: 1. Crear repositorio en GitHub 2. Configurar credenciales de Git en Databricks 3. Crear una **Git Folder** apuntando al repo 4. Trabajar en notebooks -> commit -> push

Video explicativo: Integración GitHub y Databricks https://www.youtube.com/watch?v=Z9Pq6xSQqYM

---

2. CTEs: WITH para código limpio y reutilizable

Las CTEs reemplazan las subqueries anidadas por código **legible, reutilizable y debuggeable**.

```
WITH viajes_largos AS (
 SELECT * FROM trips WHERE distance > 10
),
viajes_por_dia AS (
 SELECT DATE(pickup_time) AS dia, COUNT(*) AS total
 FROM viajes_largos
 GROUP BY DATE(pickup_time)
)
SELECT * FROM viajes_por_dia ORDER BY total DESC
```

Reglas: - Se declara `WITH` **una sola vez** al inicio - Las CTEs se separan con **comas** - Se ejecutan **de arriba hacia abajo** — una CTE solo puede referenciar CTEs declaradas ANTES - Solo viven durante el **tiempo de ejecución** de la query

Son como **variables en programación, pero para SQL**.

---

3. Window Functions: análisis sin perder granularidad

Agregan cálculos a cada fila **sin colapsar los datos** como GROUP BY.

```
FUNCION() OVER (PARTITION BY campo ORDER BY campo)
```

ROW_NUMBER() — números únicos secuenciales. Ideal para **detectar duplicados**: particionar por los campos que definen unicidad del registro y filtrar `rn > 1`.

RANK() — ranking con empates que **salta posiciones**. Notas 95, 90, 90, 85 -> posiciones 1, 2, 2, **4**.

DENSE_RANK() — ranking con empates **sin saltos**. Mismo ejemplo: 1, 2, 2, **3**.

LAG(columna) — valor de la fila **anterior**. Devuelve NULL en la primera fila. Truco: “lag” = retraso.

LEAD(columna) — valor de la fila **siguiente**. Devuelve NULL en la última fila.

LAG y LEAD evitan los **self-joins** complejos. Muy usadas en finanzas para comparaciones temporales.

---

4. PARTITION BY vs GROUP BY

GROUP BY -> colapsa filas. Resultado: **una fila por grupo**. Se pierde el detalle individual.

```
SELECT payment_type, COUNT(*), AVG(total_amount)
FROM nyc_taxi GROUP BY payment_type
-- Resultado: 2 filas (credit, cash)
```

PARTITION BY (dentro de Window Function) -> **mantiene TODAS las filas** y agrega el cálculo a cada una.

```
SELECT trip_id, payment_type, total_amount,
 AVG(total_amount) OVER (PARTITION BY payment_type) AS promedio_tipo
FROM nyc_taxi
-- Resultado: todas las filas + columna con promedio por tipo
```

GROUP BY **comprime** filas. Window Function **expande** cada fila con información extra.

---

5. Unity Catalog y Tablas Managed vs External

Unity Catalog es el catálogo unificado — el sistema de gobernanza que organiza y protege todo.

Jerarquía: Catálogo -> Esquema -> Tablas/Vistas/Volúmenes

Capacidades: Control de acceso, data masking, linaje de datos, auditoría, calidad.

Tablas Managed vs External:

	Managed	External
DROP TABLE	Borra metadatos **Y** datos	Borra solo metadatos, **datos permanecen**
Recuperación	Necesitás backup	Recreás la tabla apuntando a los datos

---

6. EDA para Data Engineering: los 6 pasos

El EDA (Exploratory Data Analysis) **no es una herramienta**. Es un proceso. Y el de un Data Engineer no es el mismo que el de un analista.

El analista explora datos que, en teoría, ya tienen calidad. Si le llega un precio negativo, va a decir que el dato está mal y te lo va a devolver para que lo corrijas. Tiene sentido: sobre eso arma métricas. Un precio negativo es como preguntar si te pagan por ir a vivir a ese alquiler. No cierra.

Nosotros recibimos el dato de la capa transaccional y lo bajamos a Bronze. En ese momento **no sabemos en qué estado está**. No podemos correr un promedio, un KPI ni un análisis de negocio encima, porque el dato nos va a mentir. A veces la query ni siquiera falla: devuelve un número y está mal. Eso es lo peor, la falla silenciosa.

Lo que sí podemos correr, antes de escribir el pipeline, es un EDA de ingeniería. No busca el insight. Busca el estado: el esquema, el count, los nulos, el tipo de dato, la cardinalidad. Es el test manual. El negocio siempre va a asumir que transformar es fácil. Si un campo que necesitan viene vacío en más de la mitad, no lo rellenamos con un número cualquiera: volvemos a la capa operativa y preguntamos si es esperado. Si no hacemos esto, la métrica ya nace mal.

El objetivo es **validar la calidad antes de pasar a Silver**. Siempre en este orden:

1. Estructura básica: `COUNT(*)` + `DESCRIBE tabla` + primeras filas. En Bronze es normal que todo quede en `STRING`. Así llegó. **2. Valores nulos:** contar nulos por columna y calcular porcentaje. Más de

50% vacío y el campo no sirve para analizar. **3. Cardinalidad:** `COUNT(DISTINCT campo)` para detectar valores inesperados. Si `tipo_operacion` tiene 20 variantes en vez de alquiler y venta, hay basura. **4. Rangos y outliers:** `MIN`, `MAX`, `AVG` para detectar inconsistencias. Recién después de castear. Sobre un string, el orden es bit a bit (`10` queda antes que `2`) y el promedio miente. **5. Duplicados:** `ROW_NUMBER() OVER (PARTITION BY campos_que_definen_unicidad)` + filtrar `rn = 1`. Si Bronze tiene 1000 filas con duplicados, Silver no puede tener 1100. **6. Análisis temporal** (si aplica)

Ingesta: archivos se suben a un **Volumen**, se leen con `read_files("ruta.csv")` y se cargan a tabla con `CREATE TABLE ... AS SELECT`.

Conceptos clave de la semana

- **GitHub y Control de Versiones integrado con Databricks:** Git es el sistema de control de versiones local. GitHub es la plataforma en la nube para hospedar repositorios.
- **CTEs: WITH para código limpio y reutilizable:** Las CTEs reemplazan las subqueries anidadas por código legible, reutilizable y debuggeable.
- **Window Functions: análisis sin perder granularidad:** Agregan cálculos a cada fila sin colapsar los datos como GROUP BY.
- **PARTITION BY vs GROUP BY:** GROUP BY -> colapsa filas. Resultado: una fila por grupo. Se pierde el detalle individual.
- **Unity Catalog y Tablas Managed vs External:** Unity Catalog es el catálogo unificado — el sistema de gobernanza que organiza y protege todo.
- **EDA para Data Engineering: los 6 pasos:** El EDA (Exploratory Data Analysis) no es una herramienta. Es un proceso. Y el de un Data Engineer no es el mismo que el de un analista.

DataDventure © 2026 — Material exclusivo para alumnos del Bootcamp